Laboratorium 3 - Kolizje, stan gry i komunikaty w WebGL2

Celem laboratorium jest przekształcenie poruszającej się sceny Road Hopper 3D z Laboratorium 2 w prostą grę z zasadami. Wprowadzimy wykrywanie zderzenia gracza z samochodem, przyznawanie punktów za pokonywanie kolejnych pasów, rozpoznawanie przegranej i wygranej oraz ponowne rozpoczęcie gry bez odświeżania strony

1. Wykrywanie kolizji - kiedy gracz dotyka samochodu?

W Road Hopperze obiekty wyglądają na przestrzenne, ale podstawowe kolizje możemy sprawdzać bez złożonej fizyki 3D. Gracz i samochody poruszają się po planszy w osiach **X** (lewo–prawo) oraz **Z** (kolejne pasy). Wystarczy wyobrazić sobie, że patrzymy na planszę z góry i zastępujemy obiekty prostokątami. Jeśli prostokąt gracza i samochodu nakładają się jednocześnie w osi X i Z, uznajemy, że doszło do kolizji. To uproszczona metoda AABB (Axis-Aligned Bounding Box), czyli sprawdzanie prostokątnych obszarów ustawionych równolegle do osi świata.



1. W katalogu *src/game* utwórz nowy plik **gameRules.js**. Nie zastępuje on *gameLogic.js*. W nowym pliku umieść poniższy kod. Pobiera on aktualną scenę, zapamiętuje jej pozycje początkowe i przygotowuje obiekt przechowujący stan rozgrywki:

```
import { rows, scene } from './sceneData.js';
export const gameState = {
   status: 'playing', // playing, lost lub won
   score: 0,
   bestLane: 0
};
const startPlayer = [...scene.player.position];
const startCars = scene.cars.map(car => [...car.position]);
const lanes = rows.map(row => row.z).sort((a, b) => b - a);
```

Zmienna gameState zapamiętuje, czy rozgrywka trwa, ile punktów zdobyto oraz do którego najdalszego pasa dotarł gracz. Tablice startPlayer i startCars przechowują **kopie początkowych współrzędnych**, aby podczas restartu można było przywrócić pierwotny układ sceny. Tablica lanes powstaje z faktycznej listy pasów w projekcie studenta, dlatego mechanizm obsługuje zarówno podstawową planszę, jak i dodatkowy pas z Lab 1.

2. W tym samym pliku, pod powyższym kodem, dopisz funkcję checkCollision():

```
export function checkCollision() {
    const player = scene.player.position;
    const halfPlayerX = 0.09;
    const halfCarX = 0.20;
    const halfPlayerZ = 0.09;
    const halfCarZ = 0.11;
    return scene.cars.some(car => {
        const dx = Math.abs(player[0] - car.position[0]);
        const dz = Math.abs(player[2] - car.position[2]);

        return dx < halfPlayerX + halfCarX &&
               dz < halfPlayerZ + halfCarZ;
    });
}
```

Math.abs() oblicza odległość między środkami obiektów, a some() sprawdza wszystkie samochody i zwraca true, jeśli co najmniej jeden spełnia warunek kolizji. Liczby 0.09, 0.20 i 0.11 są **połowami wymiarów** uproszczonych brył gracza i samochodu z projektu bazowego. Jeżeli w Lab 1 zmieniono rozmiary samochodów lub gracza, wartości te można później dopasować. Na tym etapie niczego jeszcze nie wyświetlamy – dopiero przygotowaliśmy regułę sprawdzającą zderzenia.

2. Stan gry, punkty i meta

Gra potrzebuje informacji, czy nadal można poruszać postacią i samochodami. Będziemy używali trzech wartości: playing oznacza trwającą rozgrywkę, lost przegraną po kolizji, a won dotarcie do ostatniego pasa. Wykrycie przegranej lub wygranej zatrzyma aktualizowanie ruchu, ale nie zamknie strony. Punkty przyznamy tylko wtedy, gdy gracz **po raz pierwszy dotrze do kolejnego, bardziej oddalonego pasa**. Powrót na wcześniejszy pas i ponowne wejście na już odwiedzony pas nie zwiększy wyniku.



Rys. 2. Przykładowa punktacja dla planszy z czterema pasami: 10 punktów za każdy nowy pas.

1. Pod funkcją *checkCollision()* dopisz funkcję odpowiedzialną za aktualizację zasad gry:

```
export function updateGameState() {
    if (gameState.status !== 'playing') return;
    if (checkCollision()) {
        gameState.status = 'lost';
```

```
        return;
    }
    const lane = lanes.findIndex(z =>
        Math.abs(z - scene.player.position[2]) < 0.001
    );
    if (lane > gameState.bestLane) {
        gameState.score += (lane - gameState.bestLane) * 10;
        gameState.bestLane = lane;
    }
    if (lane === lanes.length - 1) {
        gameState.status = 'won';
    }
}
```

Najpierw sprawdzamy, czy wystąpiła kolizja; wtedy stan gry zmienia się na lost. Jeżeli gracz nie zderzył się z samochodem, sprawdzamy indeks zajmowanego pasa. Wejście na nowy, dalszy pas zwiększa wynik o 10 punktów, a dotarcie na **ostatni pas w tablicy posortowanej według Z** oznacza wygraną. Ponieważ liczba pasów nie jest wpisana na stałe, meta dostosowuje się do indywidualnej planszy.



Rys. 3. Rozgrywka przechodzi z trybu playing do lost po kolizji lub do won po osiągnięciu mety.

3. Wynik i komunikaty na ekranie

Dotychczas całą scenę rysowaliśmy w WebGL2. Do prostych napisów nie potrzebujemy jednak dodatkowego Fragment Shadera ani tekstur. Wystarczy dodać element HTML nad płótnem canvas. Dzięki temu wynik i komunikat o zakończeniu gry można zmieniać bez przebudowy renderera.

1. Otwórz główny plik *index.html* projektu. **Nie usuwaj elementu `canvas` ani istniejących napisów**; przed skryptem uruchamiającym aplikację dopisz:

```
<div id="hud">
    <span id="score">Punkty: 0</span>
    <span id="status">Dotrzyj do ostatniego pasa trawy.</span>
</div>
```

2. Na końcu *src/style.css* dopisz:

```
#hud {
  position: fixed;
  top: 64px;
  left: 18px;
  padding: 10px 14px;
  background: rgba(14, 20, 29, 0.82);
  color: #f7f9fc;
  border-radius: 8px;
  font: 600 13px/1.5 system-ui, sans-serif;
  pointer-events: none;
}
#hud span { display: block; }
#status { color: #d8e4ea; font-weight: 400; }
```

Zapisz pliki i sprawdź, czy w lewym górnym rogu pojawił się napis „Punkty: 0” oraz informacja o celu gry. Na tym etapie napisy są jeszcze stałe; w następnym kroku połączymy je z rzeczywistym stanem aplikacji.

5. Połączenie zasad gry z pętlą animacji

Pętla animacji z Lab 2 wykonuje obecnie dwie główne operacje: zmienia pozycje samochodów i ponownie rysuje scenę. Teraz pomiędzy nimi dodamy sprawdzenie zasad gry. Pozostawiamy jedno wywołanie *requestAnimationFrame()* oraz istniejące renderowanie. Nie tworzymy drugiej, równoległej pętli.

1. Otwórz *src/main.js*. Na początku pliku **dodaj nowy import**, pozostawiając dotychczasowe importy *sceneData.js* i *gameLogic.js*:

```
import {
  gameState, updateGameState, restartGame
} from './game/gameRules.js';
```

Nie importuj ponownie tych samych nazw z tego samego pliku. W *gameLogic.js* nie trzeba dodawać kolejnego importu scene- pozostaje tam wcześniejsze import { rows, scene } ....

2. W *main.js*, bezpośrednio obok miejsca pobierania elementu canvas, dodaj odwołania do nowych elementów interfejsu:

```
const scoreLabel = document.querySelector('#score');
const statusLabel = document.querySelector('#status');
Poniżej istniejącej funkcji render() dodaj funkcję odświeżającą napisy:
function updateHud() {
  scoreLabel.textContent = `Punkty: ${gameState.score}`;

  if (gameState.status === 'lost') {
    statusLabel.textContent = 'Kolizja! Naciśnij R.';
  } else if (gameState.status === 'won') {
    statusLabel.textContent = 'Meta! Naciśnij R.';
  } else {
```

```
        statusLabel.textContent = 'Dotrzyj do ostatniego pasa trawy.';
    }
}
```

3. W istniejącej funkcji *frame(time)* odszukaj wywołania *updateCars(dt)* oraz *render()*. **Zastąp tylko te dwie instrukcje** następującym fragmentem, pozostawiając obliczanie dt i planowanie kolejnej klatki:

```
if (gameState.status === 'playing') {
    updateCars(dt);
    updateGameState();
}
render();
updateHud();
```

Teraz samochody poruszają się tylko podczas trwającej rozgrywki. Kiedy gracz zderzy się z pojazdem lub dotrze do mety, zmieni się gameState.status, a aktualizowanie ruchu zostanie zatrzymane. Funkcje *render()* i *updateHud()* nadal działają, dzięki czemu widoczna jest końcowa scena i odpowiedni komunikat. **Jeżeli podczas Lab 2 dodano własną pauzę**, zachowaj jej dotychczasowy warunek razem ze sprawdzaniem gameState.status, aby pauza i przegrana nie uruchamiały ruchu.

Eksperyment 1. Sprawdzenie kolizji

Uruchom grę i spróbuj wejść na pas drogi wtedy, gdy w jego pobliżu znajduje się samochód. Po kontakcie powinna pojawić się informacja o kolizji, a samochody przestaną się poruszać. Jeśli trudno uzyskać kolizję, **tymczasowo** ustaw jeden samochód w pobliżu środka drogi i nadaj mu prędkość 0, a następnie wejdź na ten pas. Po eksperymencie przywróć jego właściwe wartości. Sprawdź, czy zetknięcie z samochodem poza tym samym pasem nie powoduje przegranej.

6. Restart - ponowne rozpoczęcie gry

Restart powinien przywrócić początkowe pozycje gracza i samochodów, wyzerować wynik oraz ustawić stan playing. Nie ma potrzeby przeładowywania strony ani ponownego tworzenia VAO i VBO. Dzięki zapamiętanym wcześniej współrzędnym resetujemy wyłącznie dane sceny.

1. Wróć do *src/game/gameRules.js* i pod pozostałymi funkcjami dopisz:

```
export function restartGame() {
    scene.player.position.splice(0, 3, ...startPlayer);
    scene.cars.forEach((car, index) => {
        car.position.splice(0, 3, ...startCars[index]);
    });

    gameState.status = 'playing';
    gameState.score = 0;
    gameState.bestLane = 0;
}
```

Zapis *splice()* podmienia liczby **wewnątrz istniejącej tablicy współrzędnych**. Nie usuwa samochodów ani nie tworzy nowych modeli. Odniesienia do obiektów zapisane w projekcie pozostają takie same, a następna klatka automatycznie pokaże je we wcześniejszych miejscach.

2. W src/main.js odszukaj **istniejący** nasłuchiwacz *window.addEventListener('keydown', ...),* który steruje graczem za pomocą strzałek. Zmodyfikuj go tak, aby dodatkowo obsługiwał klawisz R. Jeżeli w Lab 2 dodano klawisz do pauzy, zachowaj jego dotychczasową obsługę. Przykładowa wersja, którą można dostosować do własnego kodu, wygląda następująco:

```
window.addEventListener('keydown', event => {
  if (event.key.toLowerCase() === 'r') {
    if (!event.repeat) restartGame();
    return;
  }
  if (!event.key.startsWith('Arrow')) return;
  event.preventDefault();
  if (event.repeat || gameState.status !== 'playing') return;
  movePlayer(event.key);
});
```

Nie dodawaj drugiego identycznego nasłuchiwacza strzałek. Popraw już istniejący, aby pojedyncze naciśnięcie nie powodowało dwóch ruchów postaci. Zapisz plik i sprawdź, czy R przywraca początkową scenę i pozwala ponownie rozegrać planszę.

Eksperyment 2. Punktacja i meta

Przejdź graczem przez kolejne pasy, starając się ominąć samochody. Za pierwszy nowy pas powinno zostać przyznane 10 punktów, za następny kolejne 10 itd. Spróbuj cofnąć się na wcześniejszy pas i ponownie przejść na odwiedzony obszar: wynik **nie powinien wzrosnąć drugi raz**. Po dotarciu do ostatniego pasa pojawi się komunikat o wygranej, a ruch pojazdów się zatrzyma. Gdy trudno przejść drogę, można tymczasowo zmniejszyć prędkość samochodów w swojej scenie, a po teście przywrócić poprzednie wartości.

Eksperyment 3. Restart i zachowanie projektu

Sprawdź działanie R zarówno po przegranej, jak i po wygranej. Wynik powinien wrócić do zera, samochody mają powrócić do punktów początkowych, a postać na pierwszy pas. Potwierdź, że dodatkowe samochody, drzewa i pasy utworzone na wcześniejszych laboratoriach nadal występują w projekcie. Nie modyfikowaliśmy ich geometrii ani funkcji rysujących.

Zadania do sprawozdania

Poniższe zadania należy wykonać samodzielnie. Polecenia opisują **oczekiwany efekt**, bez wskazywania konkretnego pliku czy funkcji do modyfikacji. Realizację można dokończyć po zajęciach.

Zadanie 1. Rozbuduj rozgrywkę o trzy życia gracza. Każda kolizja powinna odejmować jedno życie i przenosić postać na pole startowe. Gra ma kończyć się przegraną dopiero po utracie wszystkich żyć. Aktualna liczba żyć powinna być widoczna na ekranie.

Zadanie 2. Spraw, aby wraz ze zdobywaniem kolejnych punktów samochody stopniowo zwiększały prędkość, zachowując dotychczasowe kierunki jazdy. Po ponownym rozpoczęciu rozgrywki prędkości powinny wracać do wartości początkowych.

Zadanie 3. Dodaj licznik najlepszego wyniku, który pozostaje widoczny i nie znika po restarcie gry. Nowy rekord powinien być zapamiętywany wyłącznie wtedy, gdy gracz uzyska więcej punktów niż dotychczas. Wymagane jest zachowanie rekordu podczas kolejnych rozgrywek bez przeładowania strony.

Zadanie 4. Popraw wizualne defekty aplikacji – np. nachodzące się samochody, powiększenie pola gry względem postaci. Sprawdź działanie wszystkich funkcjonalności ze wcześniejszych laboratoriów.

Wymagania dotyczące sprawozdania

Sprawozdanie powinno być podzielone na poszczególne laboratoria, z wyraźnym oznaczeniem **numeru laboratorium oraz numerów wykonanych zadań**.

Dla każdego zadania należy zamieścić:

1. **Opis rozwiązania** - krótkie wyjaśnienie, w jaki sposób wykonano zadanie, np. poprzez przedstawienie zmodyfikowanego fragmentu kodu wraz z krótkim komentarzem.
2. **Zrzut ekranu**- przedstawiający efekt wprowadzonych zmian na renderowanej scenie.

Po zakończeniu trzech laboratoriów z WebGL2 należy przesłać **jedno zbiorcze sprawozdanie**, zawierające rozwiązania zadań ze wszystkich trzech spotkań.